Projeto de Disciplina — TypeScript para Desenvolvimento Front-end

Aluno: Guilherme Queiroz Ribeiro **Professor:** Kennedy Carvalho **Disciplina:** TypeScript para Desenvolvimento Front-end [26E4_2] **Data:** 09/11/2026 **Deploy (Vercel / Exímia Tech):** https://nexo.eximiatech.com.br **Repositório GitHub:** https://github.com/GuiQueirozRibeiro/nexo-finance

1. Visão Geral e Domínio do Projeto

O **Nexo Finance** (by Exímia Tech) é uma plataforma de controle financeiro inteligente que integra o ecossistema **Open Finance Brasil**, um motor de análise de gastos e detecção de anomalias inspirado no **Pierre Finance**, e operações de conta digital com emissão de **QR Code PIX e transferências via Asaas**.

O objetivo técnico central é demonstrar a construção de um módulo front-end previsível, seguro em tempo de compilação e sustentável, combinando **Clean Architecture (3 camadas + Core)**, **Princípios SOLID**, **MVVM** e **Flavors de Ambiente (`DEV` / `PROD`)** inspirados no projeto de referência `App_Nasa` . As camadas `core/` , `domain/` e `data/` são 100% agnósticas de framework visual, permitindo reuso direto na disciplina subsequente de **Angular** e na versão mobile em **React Native + Expo**.

---

2. Arquitetura em Camadas e Princípios SOLID

A estrutura em `src/` isola regras de negócio e contratos de tipos de qualquer acoplamento visual:

- `src/core/` **(Camada Core):** Implementa a mônada funcional `Either<L, R>` (`Left` e `Right`), a hierarquia orientada a objetos de falhas (`Failure` , `ValidationFailure` , `OpenFinanceFailure` , `PixGatewayFailure`), o contrato genérico `IUseCase<SuccessType, Params>` , `NoParams` , `Enums` (`AppFlavor` , `TransactionCategory` , `InstitutionCode` , `ThemeMode`), utilitários avançados de tipos (*Mapped Types*, *Conditional Types*, *Type Guards* para `unknown` e `assertNever`), o gerenciador `FlavorConfig` , o gerenciador de temas `ThemeManager` (persistência de tema `SYSTEM` / `DARK` / `LIGHT` em `localStorage` com alternância automática entre as logos `EXIMIA-FAVICON2.png` e `EXIMIA-FAVICON1.png`), o cliente `FirebaseClientManager` (autenticação Firebase Auth e sincronização Cloud Firestore no projeto `nexo-eximiatech`) e o gerenciador PWA `PwaServiceWorkerManager` .
- `src/domain/` **(Camada Domain):** Contém as entidades orientadas a objetos das 6 tabelas relacionais (`BankAccount` , `Transaction` , `CreditCardAccount` , `CostCenterBudget` , `RecurringSubscription` , `RegisteredPixKey` , `PixKeyValidator` , `PierreFinancialEngine`), tuplas financeiras, interfaces segregadas de repositório (`ITransactionRepository` , `IOpenFinanceRepository` , `IPixGatewayRepository`) e os 7 *UseCases* da aplicação.
- `src/data/` **(Camada Data):** Implementa `TransactionModel` e `BankAccountModel` (estendendo as entidades de domínio com `fromUnknown` , `toJson` , `copyWith` e `toEntity`), os *DataSources* (`FinanceLocalDataSourceImpl` persistindo as 6 tabelas no `localStorage` e sincronizando com Cloud Firestore, e `PixGatewayDataSourceImpl`) e os *Repositories* concretos, além dos Route Handlers `/api/asaas/pix` , `/api/connect-token` e `/api/open-finance/sync` .
- `src/legacy/` **(Evolução e Refatoração):** Documenta e testa a migração de um script legado de conciliação financeira em JavaScript para TypeScript estrito (`RefactoredReconciliationService`).

- **`src/presentation/` (Camada Presentation MVVM):** Contém o controlador `FinanceViewModel` em TypeScript puro e as telas minimalistas em React 19 + Next.js App Router (`/` , `/login` , `/portal` com 8 rotas dedicadas e fluxos guiados em *Wizard* passo a passo, widget Pluggy Connect oficial e `/arquitetura`).

---

3. Atendimento às 6 Etapas e 16 Itens da Rubrica

3.1. Fundamentos e Modelagem Inicial (Competência 1)

- **Tipos Primitivos, Uniões, Literais e Uniões Discriminadas (`src/domain/entities/financial-primitives.ts`):** Definidos tipos literais e uniões alinhados às regras financeiras (`TransactionType = 'INCOME' | 'EXPENSE' | 'PIX_TRANSFER'` , `PixKeyType = 'CPF' | 'EMAIL' | 'PHONE' | 'RANDOM'` , `OpenFinanceConsentStatus = 'AUTHORIZED' | 'EXPIRED' | 'REVOKED'`) e a união discriminada `PaymentMethodDetail = PixPaymentDetail | CreditCardPaymentDetail | OpenFinanceDebitDetail` processada por `describePaymentMethod(payment)` com estreitamento automático por `payment.kind` .
- **Inferência vs. Tipagem Explícita:** Tipagem explícita adotada em todas as assinaturas públicas de métodos, interfaces e retornos de *UseCases* (`Promise<Either<Failure, T>>`), enquanto a inferência do TypeScript é aproveitada em acumuladores internos (`reduce` , `filter`) e constantes imutáveis (`as const`).
- **Objetos Tipados, Tipos de Função e Sobrecarga de Funções:** Criação validada por `Transaction.validateProps(props): Either<Failure, Transaction>` , aplicação de regras via contrato funcional `FinancialFeeRule = (amount: number) => number` (`applyFinancialFeeRule`) e **sobrecarga de funções** (*Function Overloading*) em `Transaction.lookupTransaction(items, key)` (retornando `Transaction | undefined` quando buscado por `index: number` ou `readonly Transaction[]` quando filtrado por `category: TransactionCategory`).

3.2. Estruturação de Contratos e Organização (Competência 2)

- **Interfaces e Contratos (`src/domain/repositories/repositories.ts`):** Interfaces enxutas aplicando *Interface Segregation* e *Dependency Inversion*. Uso de interseção (`&`) em `DomainRecordMetadata = IdentifiableContract & AuditableContract` e tipagem de funções em `FoldCallback<Input, Output>` e `ViewStateListener` .
- **Arrays, Tuplas e Operadores Spread/Rest:**
  - `BankCoordinatesTuple = readonly [bankCode: string, branchNumber: string, accountWithDigit: string]` (ex.: `['461', '0001', '77410-9']`).
  - `BudgetThresholdTuple = readonly [idealLimit: number, maxAlertCeiling: number]` (ex.: `[1100, 1600]`).
  - `PixQrPayloadTuple = readonly [copyPasteCode: string, expiresAtIso: string]` .
  - Operador *rest* em `Transaction.calculateNetBalance(...items: readonly Transaction[]): number` e operador *spread* (`...`) no método imutável `copyWith` dos Models.
- **Classes com Encapsulamento, Visibilidade e Membros Estáticos:**
  - `BankAccount` encapsula `private _balance` e `private _consentStatus` , expondo mutações controladas via `credit()` e `debit()` que validam se o consentimento Open Finance está ativo.
  - `Transaction` utiliza `protected readonly _rawAmount` e `private _reconciled` .

- `Failure` mantém contador `private static _totalOccurrences` e subclasses com modificador `override`.
- `NoParams` e `FlavorConfig` utilizam `private constructor()` e instâncias/métodos `static`.

3.3. Abstração e Reuso com Tipos Avançados (Competência 3)

- **Proibição de `any` e Tratamento Seguro de `unknown` (`src/core/types/advanced-types.ts` e `src/data/models/`):**
  - Configurada a regra `@typescript-eslint/no-explicit-any: "error"`.
  - `safeJsonParse(raw): Either<Failure, unknown>` substitui o retorno inseguro do `JSON.parse`.
  - Os métodos `TransactionModel.fromUnknown(raw: unknown)` e `BankAccountModel.fromUnknown(raw: unknown)` refinam o dado com os Type Guards `isPlainRecord`, `hasStringField` e `hasNumberField`.
- **Operadores `keyof`, `typeof` e Asserções Seguras:**
  - `getPropertyByKey<T extends object, K extends keyof T>(target: T, key: K): T[K]` e `pickObjectKeys`.
  - `type CategoryMetadataMap = typeof CATEGORY_METADATA` em `src/core/enums/enums.ts`.
- **Generics Reutilizáveis:** `Either<L, R>`, `IUseCase<SuccessType, Params>` e `FlavorConfig.getProfileFor<F extends AppFlavor>(flavor: F)`.
- **Mapped Types, Conditional Types e Validação de Exaustividade (`never`):**
  - *Mapped Types:* `DeepReadonly<T>`, `EntityPatch<T>` e `FieldValidationMap<T>`.
  - *Conditional Types:* `FlavorGatewayContract<F>` (que alterna o contrato entre `DevGatewayContract` e `ProdGatewayContract`) e `UnwrapEitherSuccess<T>`.
  - *Exaustividade:* `assertNever(value: never): never` aplicado nos `switch` de `TransactionCategory`, `InstitutionCode`, `AppFlavor`, `PixKeyType` e `SettlementTier`.

3.4. Configuração e Ambiente (Competência 4)

- **`tsconfig.json` e `tsconfig.build.core.json`:** Habilitadas as verificações estritas (`strict: true`, `noImplicitAny: true`, `strictNullChecks: true`, `noImplicitOverride: true`, `noUnusedLocals: true`, `noUnusedParameters: true`, `noFallthroughCasesInSwitch: true`, `target: "ES2022"`).
- **Dependências de Tipos e Source Maps:** Gerenciados `@types/node`, `@types/react` e `@types/react-dom`. Habilitado `"sourceMap": true` e `"declarationMap": true`, permitindo executar `npm run build:core` para emitir os arquivos `.js`, `.d.ts` e `.js.map` na pasta `dist-core/`.
- **Enums:** `AppFlavor`, `TransactionCategory` e `InstitutionCode` integrados às regras de negócio.

---

4. Evolução e Refatoração (JavaScript → TypeScript)

No módulo `src/legacy/legacy-reconciliation.ts`, foi registrada a migração de uma rotina de conciliação de pagamentos originalmente escrita em JavaScript sem tipagem estática para a classe `RefactoredReconciliationService`:

1. **Problema 1 (Coerção de String):** No JS legado, quando `item.amount` vinha como string (`"250.50"`), o operador `+=` concatenava strings (`"1000250.50"`). Na versão refatorada, o validador `RefactoredReconciliationService.numeroValido(valor: unknown, campo: string)` refina o tipo sem uso de `any` nem `as` e retorna `Either<Failure, number>` .
2. **Problema 2 (`NaN` Silencioso):** No JS legado, `item.feeOverridePercent !== null` permitia a passagem de `undefined` , resultando em `NaN` na multiplicação da taxa. Na versão TypeScript, o estreitamento `typeof item.feeOverridePercent === 'number'` garante cálculo seguro.
3. **Problema 3 (Switch Incompleto):** Novas modalidades como `'OPEN_FINANCE_DIRECT'` não possuíam tratamento no JS legado. Na versão TypeScript, `resolveTierFeeRate(tier: SettlementTier)` utiliza `assertNever(tier)` no bloco `default` .

---

5. Registro e Avaliação Técnica do Uso de IA

Durante o desenvolvimento, ferramentas de IA foram utilizadas como apoio na estruturação inicial e refatoradas nos seguintes pontos documentados em `src/legacy/legacy-reconciliation.ts` :

Tópico	Sugestão Inicial da IA	Problema Identificado	Decisão Técnica Adotada
Desserialização JSON	`JSON.parse(raw) as TransactionModel[]`	Oculta dados corrompidos em tempo de execução e não instancia o protótipo da classe.	Implementado `safeJsonParse` retornando `Either<Failure, unknown>` validado por `fromUnknown` com Type Guards.
Flags do Compilador	Manter `"erasableSyntaxOnly": true` do template base	Impede o uso de `enum` e propriedades de construtor (`private readonly`) exigidos no domínio.	Ajustado `tsconfig.json` com `"sourceMap": true`, `"noImplicitOverride": true` e script `build:core`.
Modelagem de Entidades	Definir `BankAccount` como interface plana com `balance: number` mutável	Permite alterar o saldo diretamente na View sem checar o consentimento do Open Finance.	Modelada como classe OOP com `private _balance`, `private _consentStatus` e métodos `credit`/`debit`.
Tipos Condicionais	Tipo condicional recursivo de múltiplos níveis para rotas HTTP	Aumenta a complexidade cognitiva sem benefício prático para o domínio financeiro.	Adotados apenas tipos condicionais diretos e legíveis (`FlavorGatewayContract<F>` e `UnwrapEitherSuccess<T>`).

---

6. Como Executar e Testar o Projeto

```
npm install          # Instala dependências e pacotes @types/*
npm run typecheck    # Validação estática de tipos (tsc --noEmit)
npm run build:core   # Transpila core, domain, data e legacy para dist-core/ com
Source Maps (.js.map)
npm run lint         # ESLint estrito (regra no-explicit-any ativa)
npm test             # Executa os 19 testes unitários no Vitest (core, domain, data,
legacy e viewmodel)
npm run generate:pdf # Gera o relatório acadêmico oficial em PDF
(docs/guilhermeribeiro_typescriptparadesenvolvimentofrontend_pd.pdf)
npm run dev          # Inicia a aplicação em http://localhost:3000
```